# Experiment 09:

### Step 1: Import required libraries

### Code:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

---

### Step 2: Load the Iris dataset and convert it into a pandas DataFrame

### Code:

```
iris = load_iris()
x = iris.data
y = iris.target

feature_names = iris.feature_names
target_names = iris.target_names

df = pd.DataFrame(x, columns=feature_names)
df['target'] = y
df["target_name"] = [target_names[i] for i in y]

print("Dataset shape: ", df.shape)
df.head()
```

### Output:

```
Dataset shape:  (150, 6)
```

| | sepal length (cm) | sepal width (cm) | petal length (cm) | petal width (cm) | target | target_name |
|---|---|---|---|---|---|---|
| **0** | 5.1 | 3.5 | 1.4 | 0.2 | 0 | setosa |
| **1** | 4.9 | 3.0 | 1.4 | 0.2 | 0 | setosa |
| **2** | 4.7 | 3.2 | 1.3 | 0.2 | 0 | setosa |
| **3** | 4.6 | 3.1 | 1.5 | 0.2 | 0 | setosa |
| **4** | 5.0 | 3.6 | 1.4 | 0.2 | 0 | setosa |

---

## Step 3: Inspect dataset feature names, target classes and class distribution

### Code:

```
print("feature names: ")
print(feature_names)

print("\ntarget names: ")
print(target_names)

print("\nClass distribution:")
print(df["target"].value_counts())
```

### Output:

```
feature names:
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']

target names:
['setosa' 'versicolor' 'virginica']

Class distribution:
target
0    50
1    50
2    50
Name: count, dtype: int64
```

### Observation:

- The dataset contains four features: sepal length, sepal width, petal length, and petal width.
- The three target classes are setosa, versicolor, and virginica.
- Each class has 50 samples, making the dataset balanced.
- The dataset contains 150 samples in total.

**Step 4: Split the dataset into stratified training (80%) and testing (20%) sets**

**Code:**

```
X_train, X_test , y_train, y_test = train_test_split(x, y, test_size=0.2,
random_state=42, stratify=y)

print("training samples:", X_train.shape[0])
print("testing samples:", X_test.shape[0])
```

**Output:**

```
training samples: 120
testing samples: 30
```

**Observation:**

- The dataset was divided into 80% training data and 20% testing data.
- The training set contains 120 samples, while the testing set contains 30 samples.
- Stratified splitting maintains the class distribution in both sets.
- The random_state=42 parameter ensures reproducible results.

---

**Step 5: Standardize features using StandardScaler fitted on the training data**

**Code:**

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print(X_train_scaled[:5])
```

**Output:**

```
[[-1.72156775 -0.33210111 -1.34572231 -1.32327558]
 [-1.12449223 -1.22765467  0.41450518  0.6517626 ]
 [ 1.14439475 -0.5559895   0.58484978  0.25675496]
 [-1.12449223  0.11567567 -1.28894078 -1.45494479]
 [-0.40800161 -1.22765467  0.13059752  0.12508575]]
```

**Observation:**

- StandardScaler was used to standardize the input features.
- The scaler was fitted on the training data and used to transform both training and testing data.
- Standardization brings the features to a comparable scale, with training feature values having approximately zero mean and unit variance.
- This helps prevent features with larger numerical ranges from dominating the model.

## Step 6: Reduce standardized features to 2 dimensions using PCA

### Code:

```
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

print("Original Number of features: ", X_train.shape[1])
print("Reduced Number of features: ", X_train_pca.shape[1])

print("\nExplained Variance Ratio: ")
print(pca.explained_variance_ratio_)

print("\nTotal Explained Variance: ")
print(sum(pca.explained_variance_ratio_))
```

### Output:

```
Original Number of features:  4
Reduced Number of features:  2

Explained Variance Ratio:
[0.72677234 0.23066667]

Total Explained Variance:
0.9574390106545363
```

### Observation:

- PCA reduced the number of features from four to two principal components.
- The first principal component explains approximately 72.68% of the variance.
- The second principal component explains approximately 23.07% of the variance.
- Together, the two components explain approximately 95.74% of the total variance.
- Thus, PCA reduces the dimensionality while retaining most of the information present in the original features.

---

## Step 7: Visualize the 2D PCA-reduced training data using a scatter plot

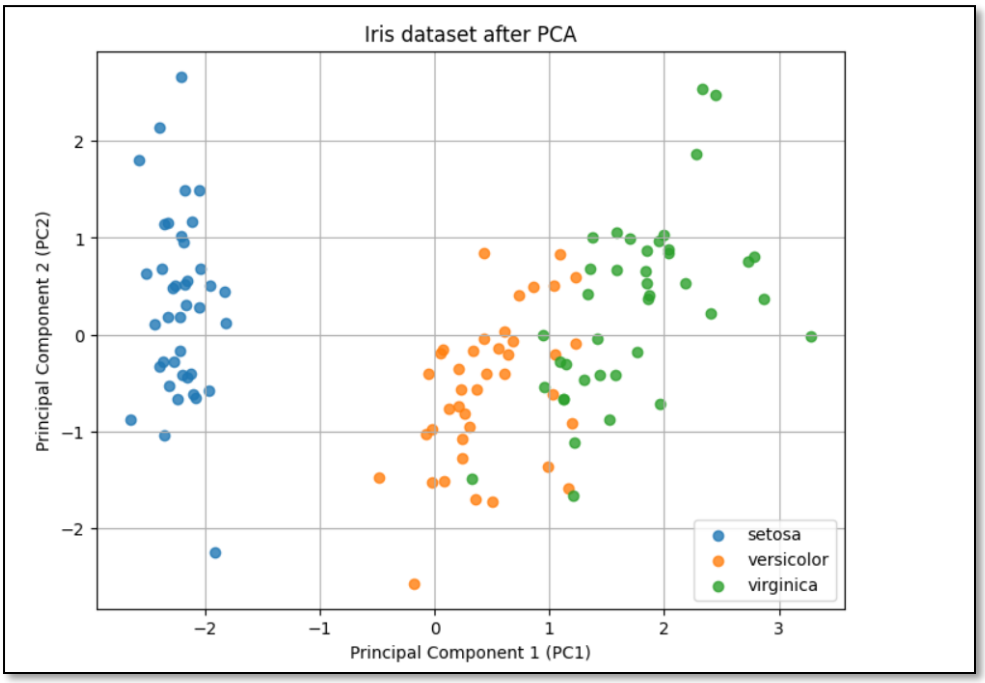
### Code:

```
plt.figure(figsize=(8, 6))

for class_value, class_name in enumerate(target_names):
    plt.scatter(
        X_train_pca[y_train == class_value, 0],
        X_train_pca[y_train == class_value, 1],
        label=class_name,
        alpha=0.8
```

```
    )
plt.xlabel("Principal Component 1 (PC1)")
plt.ylabel("Principal Component 2 (PC2)")
plt.title("Iris dataset after PCA")
plt.legend()
plt.grid(True)
plt.show()
```

**Output:**




**Observation:**

- The scatter plot displays the three Iris classes using the first two principal components.
- Setosa forms a clearly separated cluster, indicating that it is easily distinguishable from the other classes.
- Versicolor and virginica show some overlap in the plot.
- The overlap indicates that these two classes are comparatively more difficult to distinguish.
- PCA provides a two-dimensional representation of the dataset while preserving most of its variance.

---

**Step 8: Train a Logistic Regression model on PCA features and make predictions**

**Code:**

```
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train_pca, y_train)

y_pred = model.predict(X_test_pca)
print("Predicted Classes")
print(y_pred)
```

**Output:**

```
Predicted Classes
[0 1 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 1 0 2 1 2 2 2 1 0 2 0]
```

**Observation:**

- A Logistic Regression model was trained using the two PCA-reduced training features.
- The trained model was used to predict the classes of the 30 testing samples.
- The predicted output contains class labels 0, 1, and 2, corresponding to setosa, versicolor, and virginica, respectively.
- The predictions are used to evaluate the classification performance of the PCA model.

---

**Step 9: Calculate and print overall classification metrics for the PCA model**

**Code:**

```
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average="macro")
recall = recall_score(y_test, y_pred, average="macro")
f1 = f1_score(y_test, y_pred, average="macro")

results = pd.DataFrame(
    {
        "Metric": ["Accuracy", "Precision", "Recall", "F1 Score"],
        "Score": [accuracy, precision, recall, f1],
    }
)

results["Score"] = results["Score".strip()].round(4)
print(results.to_string(index=False))
```

**Output:**

```
    Metric  Score
  Accuracy 0.9000
 Precision 0.9024
    Recall 0.9000
  F1 Score 0.8997
```

**Observation:**

- The PCA model achieved an accuracy of 90.00%, meaning it correctly classified 27 out of 30 testing samples.
- Precision of 90.24% indicates the overall correctness of the positive class predictions, averaged equally across the three classes.
- Recall of 90.00% indicates that the model identified 90% of the actual samples across the classes on average.
- The F1-score of 89.97% represents the balance between precision and recall.
- These metrics indicate that the PCA model performs well, although some samples are incorrectly classified.

**Step 10: Print classification report with precision, recall and F1-score for the PCA model**

**Code:**

```
print(classification_report(y_test, y_pred, target_names=target_names))
```

**Output:**

```
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       0.82      0.90      0.86        10
   virginica       0.89      0.80      0.84        10

    accuracy                           0.90        30
   macro avg       0.90      0.90      0.90        30
weighted avg       0.90      0.90      0.90        30
```

**Observation:**

- The classification report evaluates precision, recall, F1-score, and support separately for each class.
- Setosa achieved precision, recall, and F1-score of 1.00, indicating perfect classification of its 10 testing samples.
- Versicolor achieved precision of 0.82, recall of 0.90, and F1-score of 0.86.
- Virginica achieved precision of 0.89, recall of 0.80, and F1-score of 0.84.
- The macro and weighted averages are approximately 0.90, indicating good overall performance, with most classification errors occurring between versicolor and virginica.

---

**Step 11: Generate and plot the confusion matrix for the PCA model predictions**

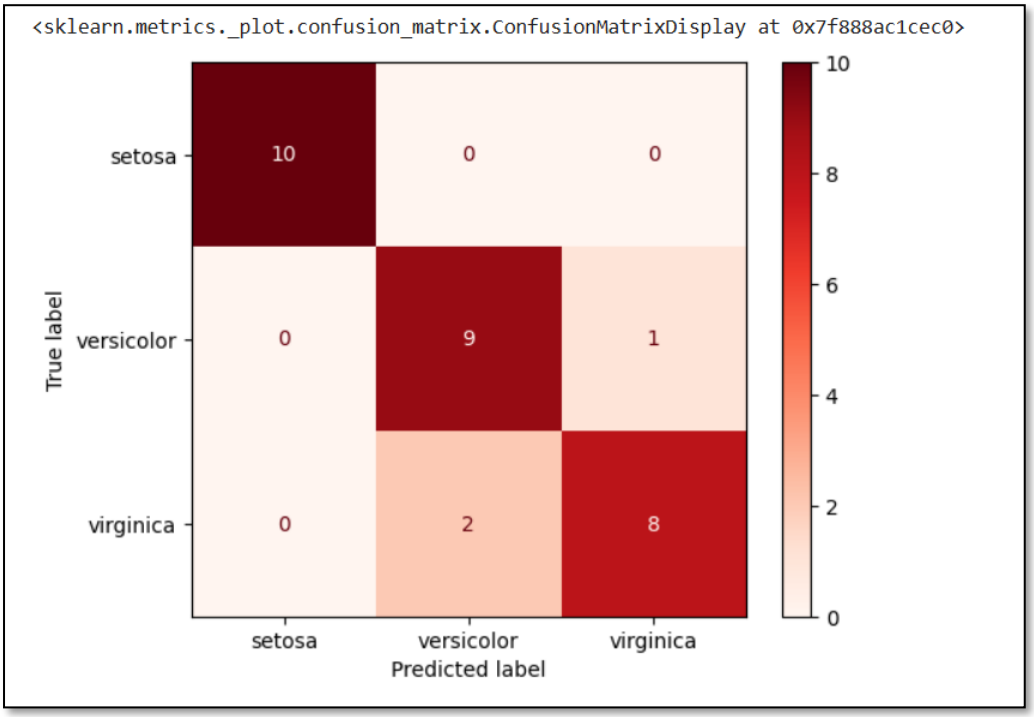
**Code:**

```
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=target_names)
disp.plot(cmap=plt.cm.Reds, values_format="d")
```

**Observation:**

- The confusion matrix compares the actual class labels with the predicted class labels.
- All 10 setosa samples were correctly classified as setosa.
- Out of 10 versicolor samples, 9 were correctly classified, while 1 was incorrectly classified as virginica.
- Out of 10 virginica samples, 8 were correctly classified, while 2 were incorrectly classified as versicolor.
- The matrix shows 27 correct and 3 incorrect predictions, confirming the model's accuracy of 90.00%.

**Output:**




---

**Step 12: Train a Logistic Regression model on the original features and predict**

**Code:**

```
original_model = LogisticRegression(max_iter=1000, random_state=42)
original_model.fit(X_train, y_train)

y_pred_original = original_model.predict(X_test)
print("Predicted Classes")
print(y_pred_original)
```

**Output:**

```
Predicted Classes
[0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 2 1 0 2 0]
```

**Observation:**

- A separate Logistic Regression model was trained using all four original features without applying PCA or standardization.
- The trained model predicted the classes of the same 30 testing samples.
- The predicted output contains class labels 0, 1, and 2 for setosa, versicolor, and virginica, respectively.
- These predictions allow the model's performance to be compared with that of the PCA-based model.

**Step 13: Print classification report with precision, recall and F1-score for original features model**

**Code:**

```
print(classification_report(y_test, y_pred_original,target_names=target_names))
```

**Output:**

```
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        10
  versicolor       1.00      0.90      0.95        10
   virginica       0.91      1.00      0.95        10

    accuracy                           0.97        30
   macro avg       0.97      0.97      0.97        30
weighted avg       0.97      0.97      0.97        30
```

**Observation:**

- The classification report shows improved performance compared with the PCA model.
- Setosa achieved precision, recall, and F1-score of 1.00, indicating perfect classification.
- Versicolor achieved precision of 1.00, recall of 0.90, and F1-score of 0.95.
- Virginica achieved precision of 0.91, recall of 1.00, and F1-score of 0.95.
- The macro and weighted averages are approximately 0.97, indicating that the original-feature model performs better overall.

---

**Step 14: Calculate and print overall classification metrics for the original features model**

**Code:**

```
accuracy = accuracy_score(y_test, y_pred_original)
precision = precision_score(y_test, y_pred_original, average="macro")
recall = recall_score(y_test, y_pred_original, average="macro")
f1 = f1_score(y_test, y_pred_original, average="macro")

results = pd.DataFrame(
    {
        "Metric": ["Accuracy", "Precision", "Recall", "F1 Score"],
        "Score": [accuracy, precision, recall, f1],
    }
)
results["Score"] = results["Score"].round(4)

print(results.to_string(index=False))
```

**Output:**

```
    Metric  Score
  Accuracy 0.9667
 Precision 0.9697
    Recall 0.9667
  F1 Score 0.9666
```

**Observation:**

- The original-feature model achieved an accuracy of 96.67%, correctly classifying 29 out of 30 testing samples.
- Precision of 96.97% indicates a high proportion of correct predictions across the classes on average.
- Recall of 96.67% indicates that the model correctly identified most of the actual samples across the classes.
- The F1-score of 96.66% indicates a strong balance between precision and recall.
- These results show that the original-feature model achieved better classification performance than the PCA model in this experiment.

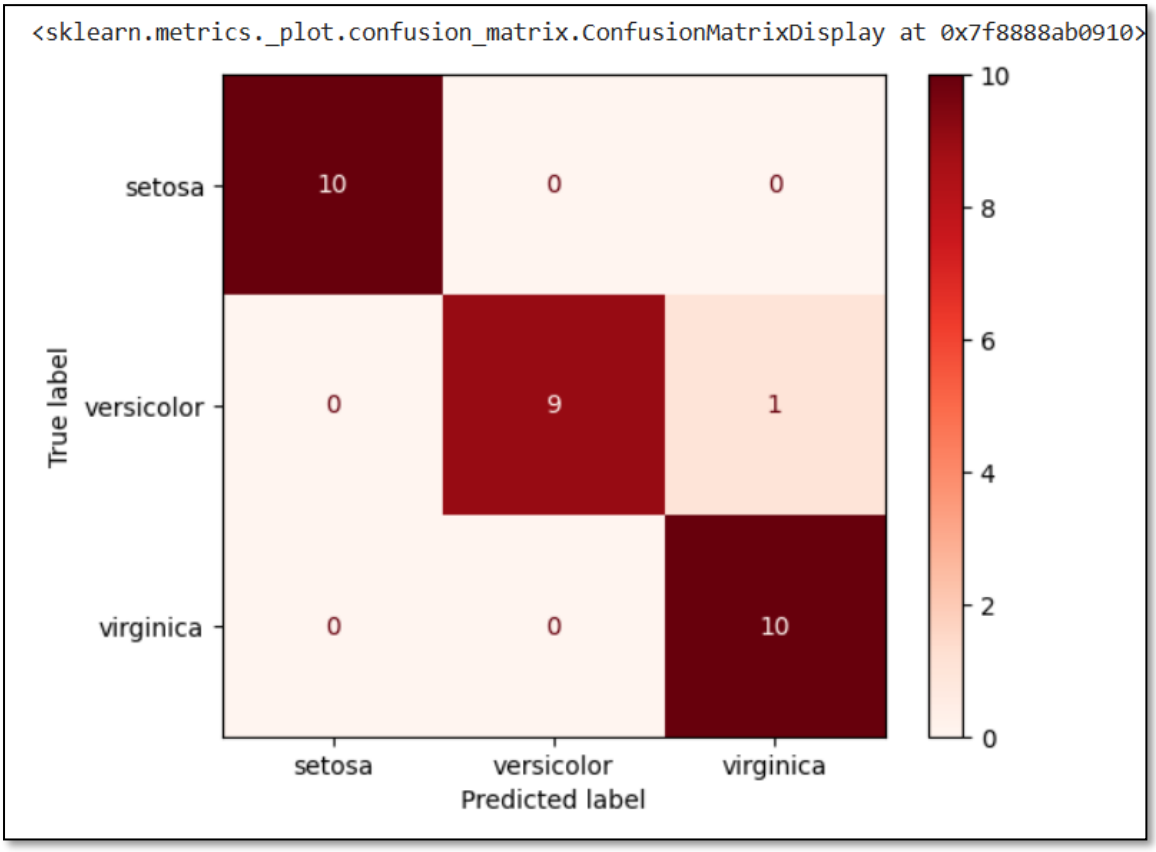
---

**Step 15: Generate and plot the confusion matrix for the original features model**

**Code:**

```
cm = confusion_matrix(y_test, y_pred_original)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=target_names)
disp.plot(cmap=plt.cm.Reds, values_format="d")
```

**Output:**



**Observation:**

- The confusion matrix compares the actual and predicted classes for the original-feature model.
- All 10 setosa samples were correctly classified as setosa.
- Out of 10 versicolor samples, 9 were correctly classified, while 1 was incorrectly classified as virginica.
- All 10 virginica samples were correctly classified as virginica.
- The model correctly classified 29 out of 30 samples, achieving an accuracy of 96.67% and making only one incorrect prediction.

---

## Comparison Between PCA and Original Features (Without PCA)

- The PCA model reduced the four original features to two principal components, retaining approximately 95.74% of the total variance. The original-feature model used all four features.
- The PCA model achieved 90.00% accuracy, while the original-feature model achieved 96.67% accuracy.
- The PCA model had a macro precision of 90.24%, recall of 90.00%, and F1-score of 89.97%. The original-feature model achieved 96.97% precision, 96.67% recall, and a 96.66% F1-score.
- The PCA confusion matrix showed three incorrect predictions, whereas the original-feature model made only one incorrect prediction.
- PCA reduces the number of input dimensions and makes visualization easier, but some information may be lost. In this experiment, the original-feature model performed better.